

TimeTuner: Diagnosing Time Representations for Time-Series Forecasting with Counterfactual Explanations

Jianing Hao, Qing Shi, Yilin Ye, and Wei Zeng, *Member, IEEE*

Abstract—Deep learning (DL) approaches are being increasingly used for time-series forecasting, with many efforts devoted to designing complex DL models. Recent studies have shown that the DL success is often attributed to effective data representations, fostering the fields of feature engineering and representation learning. However, automated approaches for feature learning are typically limited with respect to incorporating prior knowledge, identifying interactions among variables, and choosing evaluation metrics to ensure that the models are reliable. To improve on these limitations, this paper contributes a novel visual analytics framework, namely *TimeTuner*, designed to help analysts understand how model behaviors are associated with localized correlations, stationarity, and granularity of time-series representations. The system mainly consists of the following two-stage technique: We first leverage counterfactual explanations to connect the relationships among time-series representations, multivariate features and model predictions. Next, we design multiple coordinated views including a partition-based correlation matrix and juxtaposed bivariate stripes, and provide a set of interactions that allow users to step into the transformation selection process, navigate through the feature space, and reason the model performance. We instantiate *TimeTuner* with two transformation methods of smoothing and sampling, and demonstrate its applicability on real-world time-series forecasting of univariate sunspots and multivariate air pollutants. Feedback from domain experts indicates that our system can help characterize time-series representations and guide the feature engineering processes.

Index Terms—Time-series forecasting, counterfactual explanation, visual analytics

1 INTRODUCTION

Time-series forecasting is a critical research problem with a wide range of applications, including finance, economics, and weather forecasting. Accurate predictions of time-series data are vital in decision-making processes, allowing users to plan and allocate resources effectively. However, traditional methods, such as autoregressive integrated moving average (ARIMA) and its variants, are limited in modeling nonlinear and complex time-series data. Recent advances in deep learning (DL) have driven the development of more sophisticated time-series forecasting models, particularly long short-term memory (LSTM) networks. LSTM has demonstrated remarkable performance in dealing with complex time-series data, as evidenced by its success in various applications such as predicting the spread of COVID-19 [21], analyzing financial data [48], and forecasting economic trends [55].

However, DL models are often criticized for being overly complex and lacking interpretability, and their behavior is highly sensitive to the representation of time-series data. A recent study [24] has shown that appropriate data processing techniques can improve the performance of time-series prediction models without relying on overly complex DL models. In recognizing the significance of data representation, representation learning [12], which focuses on developing automated approaches for learning better data representations, has become increasingly popular in the DL domain. However, automatic approaches for representation learning alone have limitations in incorporating prior knowledge about intrinsic time-series patterns. For example, the sunspot dataset analyzed in the case study exhibits strong periodic patterns of approximately 13 months, which are not readily apparent in the feature space. For multivariate time series, automated approaches alone are inadequate for uncovering interactions among variables [44]. There is a growing need for interactive solutions to increase user en-

gagement in exploring the impact of different transformation methods on predictions, analyzing outliers, gaining more intuitive explanations, and subsequently selecting appropriate transformation methods.

Meeting this need is challenging due to two main reasons. Firstly, current evaluation for time-series forecasting is typically limited to single numeric metric, such as root mean square error (RMSE) for the training/testing dataset as a whole, which overlooks the uniqueness of individual sliding windows and does not explicitly reveal which parts of the data representations are responsible for model behavior and resulting predictions [61]. Moreover, the lack of interpretability of DL models hinders users from understanding how different transformation methods affect predictions, preventing them from mining insights and understanding the reasons behind them. Secondly, applying DL models to time-series forecasting requires considering a large number of possible data representations and vast volume of time-series data, even for a single representation. The sliding window mechanism in data transformation also leads to a significant overlap in data. Existing time-series visualization methods cannot address these issues adequately.

To overcome these challenges, this work contributes a visual analytics framework, namely *TimeTuner*, leveraging counterfactual explanations to explain predictions for individual sliding windows, along with interactive time-series visualizations that are scalable to large-volume time series and multiple variables. *TimeTuner* comprises two-stage methods. Firstly, in the *forecasting and explaining* stage, raw time-series data are transformed into different representations using various transformation methods. The representations are then passed to an LSTM model, and counterfactual explanation metrics, including SHapley Additive exPlanations (SHAP) and correlation, are derived. Secondly, in the *interactive exploration* stage, we develop a multiple-view visualization system that reveals the associations among variables, representations, and predictions from various perspectives. Specifically, we propose a new design of juxtaposed bivariate stripes that can simultaneously depict bivariate representation and prediction metrics of multiple time-series representations. We also leverage partition-based correlation matrices [44], horizon graphs [28], and other designs to address diverse analytical tasks. To demonstrate the general applicability of *TimeTuner*, we instantiate the system with two transformation methods of smoothing and sampling, and apply them to both univariate and multivariate time-series forecasting tasks on two real-world time-series datasets. The case studies, along with feedback from domain experts, illustrate the effectiveness of our system.

In summary, our work makes the following contributions:

- Jianing Hao, Qing Shi, Yilin Ye, and Wei Zeng are with the Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China. Yilin Ye and Wei Zeng are also with the Hong Kong University of Science and Technology, Hong Kong SAR, China. E-mail: {jhao768@connect., brantqshi@, yye@connect., weizeng@}hkust-gz.edu.cn
- Wei Zeng is the corresponding author.

Manuscript received 31 March 2023; revised 1 July 2023; accepted 8 August 2023.
Date of publication 26 October 2023; date of current version 21 December 2023.
This article has supplementary downloadable material available at <https://doi.org/10.1109/TVCG.2023.3327389>, provided by the authors.
Digital Object Identifier no. 10.1109/TVCG.2023.3327389

- We develop *TimeTuner*, a general visual analytics framework that integrates counterfactual explanations with interactive visualizations to diagnose the interactions among variables, representations, and predictions in time-series prediction. *TimeTuner* is instantiated with two examples of common transformation methods to demonstrate the effectiveness of the framework.
- We propose a new design of juxtaposed bivariate stripes that provides a scalable visualization for comparing and exploring different time-series representations and corresponding predictions.
- We conduct two cases on univariate and multivariate datasets using *TimeTuner*. Our analysis uncovers valuable insights, such as the sensitivity of predictions on abnormal events and the association between performance metrics and explanation metrics. These insights provide useful hints for selecting appropriate transformation methods and explaining why models perform poorly.

2 RELATED WORK

Time-Series Forecasting. Traditional time-series forecasting methods focus on parametric models informed by domain knowledge, such as auto-regression (AR), ARIMA [14], and exponential smoothing [26]. In recent years, DL has gained increasing popularity in time-series forecasting. Among various DL models, recurrent neural networks (RNNs) (e.g., [37, 47]) have gained significant attention due to their capability to remember past information and model temporal dependencies. However, when learning long-term dependencies, RNN architecture can suffer from gradient explosion and disappearance. LSTM was developed to address these limitations [38]. Nowadays, a plethora of works based on LSTM and its variants have been applied to time-series forecasting in various fields such as petroleum industry [48], medical treatment [21], and financial market [55].

DL-based time-series forecasting is a complex process that requires careful consideration of data processing, feature engineering, and model development. There has been a focus on developing advanced DL models, along with various visualization systems (e.g., [33, 40, 52, 57, 64]) developed to facilitate sequence model understanding and improvement. However, challenges including limited improvement and generalizability issues remain unsolved [24]. Few efforts have been dedicated to data processing and feature engineering, which can also lead to improved predictions. While automatic approaches such as representation learning [12] are popular, they have been criticized for their incompetence to incorporate prior knowledge and identify local interactions between variables [44]. Therefore, there is a need for an interactive visualization tool to analyze the impact of different transformation methods for time-series data, which is essential for accurate forecasting. This work aims to fulfill this need by combining counterfactual explanations with visual analytics to meet diverse requirements.

Counterfactual Explanations. DL models are generally regarded as ‘black box’ that is difficult to obtain human-comprehensible explanations directly from the models. To address this challenge, a variety of explainable artificial intelligence (XAI) techniques have been proposed, which can be divided into two types: intrinsic interpretable models and post-hoc interpretation models [7]. Intrinsic interpretable models refer to models that have inherent interpretability, such as decision trees and linear regression. However, the trade-off between interpretability and accuracy often hinders the usability of this class of methods [50]. Post-hoc model explanation techniques are typically model-agnostic, with some explaining a model’s decisions by calculating feature attributions or through case-based reasoning, and others providing counterfactual explanations to reveal what changes should have been made to the input instance to alter the outcome of an AI system [35, 41, 59].

Counterfactual explanations are hypothetical examples that explore the relationship between a model’s input and output. By importing different inputs that could lead to different outputs, these hypothetical examples can help users understand how the model arrived at its predictions and understand the DL pipeline. The *What-If Tool*, developed by Google [61], supports analysis on both single data and the entire dataset, making it a versatile tool for exploring counterfactual explanations. As counterfactual explanations do not require prior knowledge of deep

learning, they are user-friendly to general users and have been applied in multi domains [13, 30, 43]. In the context of forecasting, counterfactual explanations can aid in determining which processing methods can be used to optimize future outputs (e.g., [66]). Combining them with interaction and visualization designs can help explore and interpret counterfactual explanations [64]. For example, DCDE [19] provides a set of interactions that enables users to customize the generation of counterfactual explanations to gain further insights.

In line with this research direction, our work employs counterfactual explanations for time-series forecasting, along with interactive visualizations that enable users to explore these explanations interactively. This is nevertheless a challenging task because DL-based time-series forecasting involves components in various aspects, such as multivariate variables, diverse transformation methods, and large-volume time-series data. A comprehensive exploration would require scalable time-series visualizations that can handle these tasks.

Time-Series Visualization. Appropriate visualizations are required to explore and analyze multivariate and large-volume time-series data [9]. Line graph is a common technique for visualizing time series, allowing users to detect data trends and other patterns [60]. However, line graph can easily become unwieldy for complex time-series data. Variations on line graphs, such as horizon graphs [49], stacked graph [17], and braided graphs [29] have been developed to address the scalability issue. These visualizations utilize vertical position or area to encode data values, which however, are constrained to the limited display space and human perception of minimal areas. Other approaches are more scalable by mapping time-series data to pixel-wise visual elements and using color to encode their values, such as heatmaps [27, 34], sparkboxes [15], and heat stripes [16]. Calendar-based visualization [45, 58] are also popular for visualizing periodic time-series data.

In addition to visualization techniques, interaction methods can be employed to address the scalability problem in exploring large-volume time-series data. Overview+detail and focus+context are common interaction methods that help users examine time-series data with rich details while maintaining context [31, 32, 53, 63]. Zooming is also a widely used technique, especially for exploring large time scales. For example, TimeZoom [23] enables zooming to explore the arbitrary granularity of time units, while PlanningLines [10] allows zooming and panning for examining fine-grained plans with large time scales.

Our work aims to investigate the impact of various transformation methods on predictions through counterfactual explanations and an intuitive visual interface. Given the extensive range of transformation methods and the potential overlap issues that arise from the sliding window mechanism, developing effective visualization techniques poses a significant challenge. To overcome this obstacle, we propose a novel design of juxtaposed bivariate stripes, which is a modified version of heat stripes that offers the benefits of visualizing large-volume time series while also presenting bivariate representation and prediction metrics simultaneously using value-suppressing uncertainty palette (VSUP) [22]. Brushing and sorting supported in this design enable easy comparison and exploration of various representations and predictions.

3 BACKGROUND, TASKS, AND SYSTEM OVERVIEW

This section introduces the research background (Sect. 3.1), followed by analytical tasks (Sect. 3.2), and system overview (Sect. 3.3).

3.1 Background

Time-series forecasting is a fundamental problem in machine learning, particularly with the growing utilization of deep learning approaches. These tasks can be categorized as single-step or multi-step, depending on the prediction horizon, and as univariate or multivariate, based on the number of attributes. Since single-step forecasting is less common in practical applications, this work primarily focuses on multi-step time-series forecasting for both univariate and multivariate data.

Mathematically, this work considers a time-series dataset with T time steps and k variables, denoted as $\{X_t | t \in \{1, \dots, T\}\}$, where each $X_t \in \mathbb{R}^k$ denotes a set of observations at time step t . For univariate time-series forecasting, $k = 1$, while for multivariate time-series forecasting, $k > 1$. Our goal is to predict the values of the target variable, denoted

by y , for time steps $T + 1$ to $T + \Delta T$, based on the values of all k variables at time steps 1 to T . Here, y is the same variable with the input time series for univariate forecasting or one of the variables in the input time series for multivariate forecasting. ΔT is the fixed forecasting length that varies for different tasks. We can represent this as follows: $\{y_{T+i}\}_{i=1}^{\Delta T} = f(\{X_t\}_{t=1}^T)$, where f is the mapping function, which can be learned using various machine learning techniques, such as RNNs, LSTMs, or other deep learning models, depending on the task's complexity and the data's nature.

3.2 Analytical Tasks

Based on our prior experience in utilizing LSTMs and a comprehensive review of existing research on representation learning for time series, we have observed that model developers are becoming increasingly dissatisfied with using a single overall metric for model performance. Instead, they favor emerging counterfactual explanation techniques supporting “what-if analysis” for instance. We first surveyed existing design studies for XAI, especially those using counterfactual explanations [8, 19, 52, 56, 61], which emphasize the relationship between a model's input and output, *i.e.*, time-series representations and predictions. Next, we referred to existing studies on time-series forecasting, and identified key requirements such as identifying interactions between variables [18, 42, 44], which is essential for representation learning [12].

In this way, we summarize three-level analytical tasks: *T1) variable-level*, *T2) representation-level*, and *T3) prediction-level exploration*. Variable-level exploration involves examining the details of variables and identifying associations among them.

T1.1 Examine the details of variables. For effective counterfactual explanation, users need to inspect the details of variables, such as outliers of a variable and how different transformation methods affect them.

T1.2 Identify associations among variables. In multivariate time-series forecasting, it is challenging to isolate a single variable and explain predictions independently. Therefore, users must first comprehend the associations between input variables to understand how these variables jointly influence the predictions.

Representation-level exploration is paramount, which involves comparing different representations and analyzing their associations with variables and predictions.

T2.1 Compare different representations. It is necessary to provide users with an *overview* of all the available representations to enable efficient comparison, identify problematic representations, and select appropriate ones. To facilitate a deeper understanding of the varying quality of different representations, it is desirable to allow users to zoom in to *details* of each representation at the finest level, namely specific windows with salient values, which may be the root cause of unexpected model performance.

T2.2 Analyze the association between representations and variables. Users need to examine the relationship between variables and representations, particularly in multivariate time-series forecasting. Understanding how each variable is correlated with, emphasized, or neglected by certain representations can help explain why certain representations lead to specific results.

T2.3 Analyze the association between representations and predictions. One key task for users seeking to explain models in terms of representations is to understand the effects of different representations on predictions. This involves analyzing the correlation between representations and predictions, focusing on localized correlations that can provide counterfactual explanations.

Prediction-level exploration aims to overview the overall patterns and detect outliers of predictions.

T3.1 Overview the overall patterns of predictions. Overviewing the status of the predictions, including changes of the predictions over time, and corresponding evaluation metrics, is a prerequisite for downstream tasks such as outlier detection.

T3.2 Detect the prediction outlier. To diagnose model failures such as underfitting and overfitting problems, it is important to assist

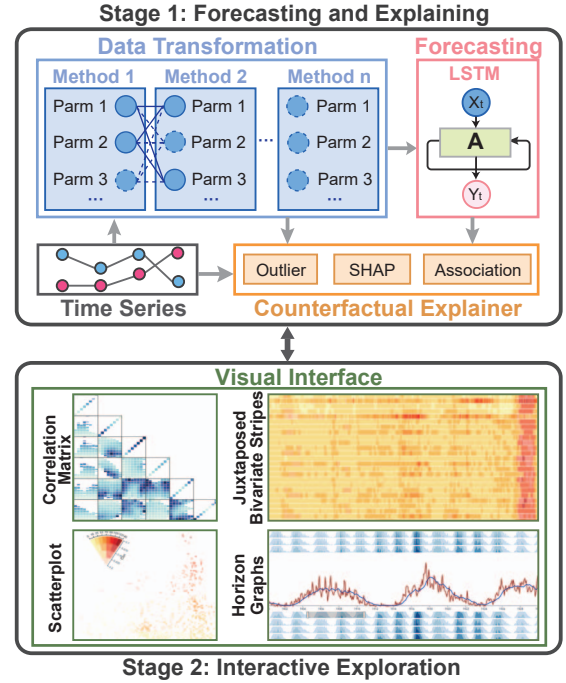


Fig. 1: Overview of the *TimeTuner* workflow, which mainly consists of four modules in two stages: 1) data transformation, 2) forecasting, and 3) counterfactual explaining in the *forecasting and explaining* stage, and 4) visualization system in the *interactive exploration* stage.

users in quickly discovering outliers in the predictions for the subsequent counterfactual explanation. These outliers could reveal potential issues with the model or data and guide further analysis and refinement of the model.

3.3 System Overview

An overview of *TimeTuner* is shown in Figure 1, which mainly consists of four modules in two stages: 1) *data transformation*, 2) *forecasting* and 3) *counterfactual explaining* in the *Forecasting and Explaining* stage, and 4) *visualization system* in the *Interactive Exploration* stage.

In the first stage, raw time series are passed into the *data transformation* module, yielding a variety of time-series representations. The module can contain different transformation methods and optional parameters for time series. These methods affect properties of time-series representations, including localized correlations, stationarity, and granularity, and consequently affect the forecasting results. Next, the representations are passed to the *forecasting* module, where we choose the LSTM network as the forecasting model, a successful architecture ubiquitously applied to time-series forecasting. Finally, the *counterfactual explaining* module combines outlier detection, feature importance, and association analysis methods on raw and transformed time series, together with forecasting results, to support the counterfactual explanation of the forecasting model.

In the second stage, time-series representations, forecasting results, and explanation metrics are visualized in a coordinated multiple-view (CMV) system that mainly consists of the following: 1) *Variable Inspector View* presenting variable distribution and multi-variable correlations with a partition-based correlation matrix of Mosaic plots; 2) *Representation View*, displaying bivariate representation and prediction metrics for multiple representations with juxtaposed bivariate stripes color coded using VSUP; 3) *Prediction Comparator View*, depicting prediction distribution from the perspectives of explanation metrics using scatterplot; 4) *Temporal View*, visualizing time-series data details with horizon graphs. A set of interactions are also provided, allowing users to explore the relationships among variables, representations, and predictions. The visualization modules are implemented in D3.js [2] and integrated using Vue.js [5], with Python Flask [1] in the backend.

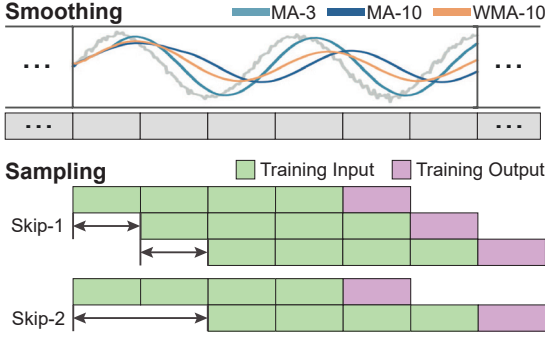


Fig. 2: The data transformation includes two steps: 1) *smoothing* using *MA* and *WMA* to remove noises and generate smooth time series, and 2) *sampling* with different skip lengths.

4 TRANSFORMATION, FORECASTING, AND EXPLANATION

This section introduces methods of data transformation (Sect. 4.1), models used for time-series forecasting (Sect. 4.2), and metrics used for counterfactual explanation (Sect. 4.3), utilized in this work.

4.1 Data Transformation

Raw time-series data often contain deficiencies (*e.g.*, outliers) that can significantly impact the accuracy of forecasting results. It is necessary to transform the data and generate proper time-series representations before feeding them into a forecasting model. In this work, *TimeTuner* is instantiated with two transformation methods—smoothing and sampling, which are commonly used transformation methods for time-series data. We aim to examine the impact of these two techniques on the forecasting results. Specifically, we compare various commonly used smoothing and sampling methods as follows.

- **Smoothing.** Smoothing is a widely-used technique to reduce or eliminate fluctuations in time series, which can increase the robustness of forecasting models against outliers. This work evaluates two popular smoothing methods: *Moving Average (MA)* and its variant *Weighted Moving Average (WMA)*. A simple moving average is calculated as the unweighted mean of the previous m time-series data points: $MA_m = \frac{1}{m} \sum_{i=n-m+1}^n X_i$, where m denotes the number of data points to be averaged. Instead of unweighted mean, *WMA* assigns varying weights to the time-series data at different positions. A typical example is to decrease weights in arithmetic progression when analyzing financial data. The formula for *WMA* is defined as follows: $WMA_m = \frac{2}{m \times (m+1)} \sum_{i=n-m+1}^n X_i \times (n+1-i)$.

Both *MA* and *WMA* use a smoothing parameter, denoted by m , to control the degree of smoothing. A larger value of m results in a smoother effect. Users can choose different values of m for different time series data and prediction tasks based on their prior knowledge.

- **Sampling.** After smoothing, a training dataset is needed for constructing DL-based forecasting models. To make use of all available data, we employ the sliding window (SLICS) technique for sampling time series. The choice of an appropriate step size is crucial. The step size determines the granularity of the sampled time-series data, and impacts the accuracy and precision of the forecasting results. As shown in Figure 2 (bottom), a smaller step size generates more windows, providing a detailed picture of the time-series data. However, these windows may exhibit more variations that are difficult to learn. Conversely, a larger step size may provide a more general view of time-series trends and patterns but may miss important changes or fluctuations between data points. This work considers each sliding window as a time slice of the minimum analysis unit.

Each combination of smoothing and sampling methods yields a unique time-series representation, denoted as $\{X_t^s | t \in \{1, \dots, \frac{T}{s}\}\}$, where X_t^s is the sliding window at position t , and s is skip length utilized in the corresponding transformation method.

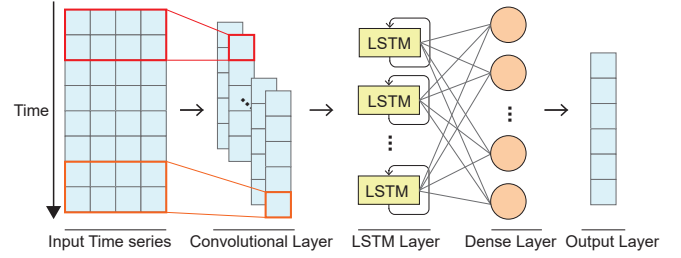


Fig. 3: The simplified architecture of the LSTM model used for time-series forecasting in this work.

4.2 Time-series Forecasting

Before applying the DL forecasting model, we perform the following tests on each time-series representation $X^s = \{X_t^s\}$:

- **Augmented Dicky-Fuller (ADF).** The ADF test checks stationarity in time-series data, assuming that the time series follows a random walk model with drift [20]. Additional transformations are needed to achieve stationarity if the test fails.
- **Autocorrelation Function (ACF).** The ACF value indicates the strength and nature of temporal dependencies between data points [65]. The formula for calculating ACF at lag m for a stationary time series X^s is as follows:

$$ACF(X^s, m) = \frac{\sum_{t=1}^n (X_t^s - \bar{X}^s)(X_{t-m}^s - \bar{X}^s)}{\sum_{t=1}^n (X_t^s - \bar{X}^s)^2}, \quad (1)$$

where $\bar{X}^s$ denotes the mean value of X^s .

We adopt a simplified LSTM model as illustrated in Figure 3, implemented using the Keras framework [3]. The input matrix is a 3D array of shape (samples $\times$ time steps $\times$ features), where samples are the number of sliding windows; time steps correspond to the length of the training data within each window; features denote the number of input variables. We first apply the convolutional layer to detect and extract features from the input data. The generated feature maps are then fed into the LSTM layer, which learns to identify important patterns and make predictions based on them. Next, a dense layer and an output layer are used to make predictions using the learned features. To focus on the impact of different transformation methods rather than other experimental settings, we adopt consistent experimental settings for all transformations. Specifically, we choose commonly used setups of 80:20 train-test data split ratio, and 100 epochs that reach converging state for the sunspots and air pollutants cases, to train a model for each time-series representation. We choose a commonly used evaluation metric, *RMSE* [62], that can be applied to evaluate the performance of both a time-series representation and individual SLICS.

4.3 Counterfactual Explanation

To provide actionable guidance to end-users in a user-friendly way, *TimeTuner* supports exploratory analysis of the impact of transformation methods on predictions by combining the counterfactual explanation metrics with a set of interactions to enable users to customize the generation of counterfactual explanation to find more insights and help their decision-making process. In this section, we introduce the two metrics leveraged for the counterfactual explainer, including:

- **SHapley Additive exPlanation value (SHAP)** [6, 39] reflects the influence of the features in each sample. Suppose the i th sample is x_i , the j th feature of the i th sample is $x_{i,j}$, the model's predicted value of the i th sample is y_i , and the baseline of the entire model is y_{base} (usually the mean of the target variable for all samples), then the SHAP value $f(x_{i,j})$ of $x_{i,j}$ satisfies the following equation:

$$y_i = y_{base} + f(X_i, 1) + f(X_i, 2) + \dots + f(X_i, k), \quad (2)$$

where $f(X_i, j)$ is the contribution value to y_i . $f(X_i, j) > 0$ indicates that the feature j has a positive effect on the predicted value, while $f(X_i, j) < 0$ indicates negative effect.

- **Pearson correlation coefficient (CORR.)** [4] measures how strong the relationship between the predictions and sample is [11]. The Pearson correlation takes as input two data matrices X and Y :

$$CORR. = \frac{n \times \sum XY - \sum X \times \sum Y}{\sqrt{(n \times \sum X^2 - (\sum X)^2) \times (n \times \sum Y^2 - (\sum Y)^2)}}, \quad (3)$$

where $\sum XY$ is the sum of the products of samples and prediction values. CORR. ranges in $[-1, 1]$, where 0 indicates no correlation and $+/-1$ indicates perfectly positive/negative correlations.

5 VISUALIZATION DESIGN

We develop a visual analytics system to address the analytical tasks (Sect. 3.2). The system is expected to meet the following design criteria.

DC1. Coordinated Views. Various transformation methods generate representations distinct from each other, yielding diverse variable-, representation-, and prediction-level metrics. To comprehensively reveal the associations between input representations and output predictions, coordinated multiple views (CMVs) that support visual analytics from multiple joint perspectives would meet the requirement.

DC2. Overview + Details. The visual analytics system should provide users with a clear overview of the input representations and output predictions while enabling detailed exploration as needed. This can be achieved by combining aggregated visualizations, which provide a high-level overview, with unit visualizations that offer more granular details.

DC3. Flexible Interactions. Our system must offer rich interactions for flexible exploration. Sorting time-series representations using various metrics can facilitate comparison while selecting and filtering operations can promote a more comprehensive *overview + details* exploration.

Based on these design criteria, we design a CMV system that primarily incorporates five modules (Sects. 5.1–5.5), integrated with a set of interactions (Sect. 5.6) to facilitate system exploration.

5.1 Profile View

The view allows users to upload a time-series dataset for exploration and presents an overview of available representations ($T2.1$). Statistics, including a number of variables, stationarity, and periodicity of the chosen dataset, are presented. Users can also select one or multiple transformation methods and determine their parameters. For example, in Figure 4 (A), the user selects two commonly adopted transformation methods of smoothing and sampling, and chooses time intervals of one month (1), one season (3), half a year (6), and meaningful periods (13) for sunspots advised by SILSO¹.

The derived representations are presented in a table with sortable bars for users to have an overview of all representations and quickly identify representations of interest. The table presents each representation as a row and shows its training error, validation error, and ACF value encoded by bar lengths. Clicking on a row of interest will highlight the corresponding representation in *Representation View* (Figure 4 (D)). Users can sort the table according to different metrics. In Figure 4 (A), the table shows that the representation ‘MA-3/Sk-1’ (which stands for Moving Average with $m = 3$ and skips with step size 1) has a large training error, indicating an abnormal representation for sunspot forecasting tasks. Upon selection, the corresponding representation is highlighted in *Representation View* (Figure 4 (D)).

5.2 Variable Inspector View

Variable Inspector View (Figure 4 (B)) is designed to support the exploration of relationships between a feature space of continuous or discrete input variables and the target variable ($T1.2$, $T2.2$). Inspired by [44], we provide a partition-based correlation matrix that partitions a variable into regions using uniform partitioning, and uses Mosaic plots to

overview local relationships of pair-wise variables/representations. The correlation matrix consists of multiple plots, each containing several rectangular regions. The regions are colored based on the local relation metrics, with dark colors indicating a stronger correlation while no color for no correlation. The view presents different information for univariate and multivariate time-series data as shown in Figure 5.

- For univariate data, this view shows the data distribution under different smoothing methods. First, each time step in the time series X_t is represented as a point in the scatterplot with its x & y corresponding to the smoothed time series. Then we aggregate the scatterplot into Mosaic plots which are uniformly partitioned into different regions, with the color of each region encoding the average values within the region. When the data on X - and Y -axes are the same representation, only regions on the diagonal are colored. The diverse distributions reflect the effects of different smoothing methods. For instance, colors in the ‘Raw Data’ and ‘WMA-13’ plot (Figure 5 (left)) shift towards the upper right part, indicating that the time-series data smoothed by the ‘WMA’ method would retain forward information making the data distribution shift.
- For multivariate data, the view visualizes correlations of all input variables that are ranked from top to bottom by variable importance. Within each plot, different input variables are represented on axes, while each region is color-coded by the value of the target variable. By examining regions in each plot, users can mine the relationship between input variables and the target variable, along with the relationship between pairs of variables and the target variable. As shown in Figure 5 (right), the plots represent the relationship between ‘Temperature’ and ‘Relative Humidity’. The dark regions indicate high values of the target variable ‘PM2.5’, while shallow regions indicate small ‘PM2.5’ values. The marked regions where ‘Temperature’ is high and ‘Relative Humidity’ is low indicate special patterns between ‘Temperature/Relative Humidity’ pairs and PM2.5 values.

5.3 Temporal View

Temporal View (Figure 4 (C)) is designed to visualize the details of different time-series representations for univariate data ($T2.1$) and variables for multivariate data ($T1.1$). It consists of two parts: horizon graphs for the context information and a line chart for the details.

- For the context information, each variable of the input time-series data is visualized as a 4-layer horizon graph [28] to help users get a picture of data distribution. The horizon graph is a combination of area graph and histogram, allowing for more efficient use of screen real estate. As an example, it can be found that the number of sunspots dramatically increases in the 1960s (dark blue parts).
- For the details, we use a multiple-line chart with colors to encode different representations/variables selected by users. The selected time period is highlighted in the corresponding horizon graph. The line chart allows users to identify and analyze abnormal representations/variables by examining the details.

5.4 Representation View

Representation View (Figure 4 (D)) is used to compare different representations ($T2.1$) meanwhile depicting associations between representations and predictions simultaneously ($T2.3$). A key challenge here is to be scalable to too many time-series representations by different transformation methods and different parameters, each coupled with bivariate representation- and prediction-level metrics. To address the challenge, we propose a novel design of juxtaposed bivariate stripes, with each time-series representation depicted as a strip and multiple representations juxtaposed from top to bottom. All stripes share the same timeline with that in *Temporal View*. For instance, based on the skip length given in *Profile View*, the view in Figure 4 (D) provides a total of 28 representations (seven smoothing alternatives $\times$ four sampling alternatives for the sunspot data). For each slicing window in a representation, we compute its explanation metric (CORR. / SHAP value) as the first dimension and its performance metric (RMSE) as the second dimension. The two-dimensional values of each window

¹ <http://www.sidc.be/silso/datafiles>



Fig. 4: The interface of *TimeTuner*. *Profile View* (A) provides information about the input dataset and an overview of all representations. *Variable Inspector View* shows the localized distribution of different representations of univariate time series (B), helping users compare different smoothing methods. *Temporal View* (C) presents global and local information of multiple representations/variables using horizon graphs. *Representation View* (D) displays different time-series representations for users to explore, which shares the same color coding with *Prediction Comparator View* (E).

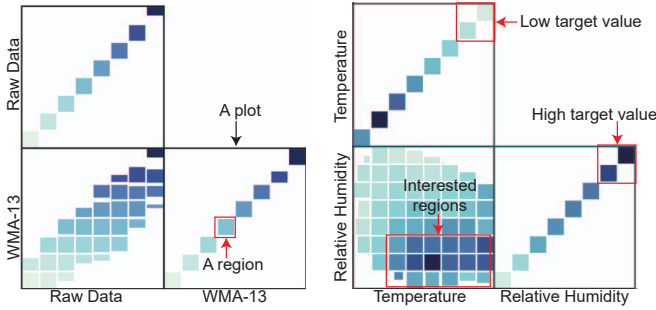


Fig. 5: Left: correlation between 'Raw' and 'WMA-13' representations for univariate time-series data. Right: correlation between pairs of variables 'Temperature' and 'Relative Humidity' and the target variable 'PM2.5' for multivariate time-series data.

are encoded using a VSUP [22]. As Figure 4 (E1) shows, VSUP is in a wedge shape. We divide the values of the first dimension into eight ranges, while the values of the second dimension are divided into four ranges. The use of VSUP encourages users to conduct more careful inspections of error-high slices and compare them. We also support the exploration of changes in a single-dimension metric, by selecting the wanted metric through the drop-down box. The VSUP palette will be changed to a sequential colormap correspondingly.

To enhance the view's scalability for large time-series data with numerous windows exceeding screen pixel capacity, we employ a two-stage optimization: First, as discussed in Sect. 4.1, because the skip length is always smaller than the span of a sliding window, the windows of a given representation have serious overlap on the time axis. Therefore, if each window is represented by rectangles equal to its original length like in a normal heatmap, the colors of each sample will blend together and dilute the salient window. To address the problem, we adjust the rectangles' length to fit the particular representation's skip length. Each window is represented as a small rectangle aligned to the start of its corresponding sub-window. This ensures that each skip

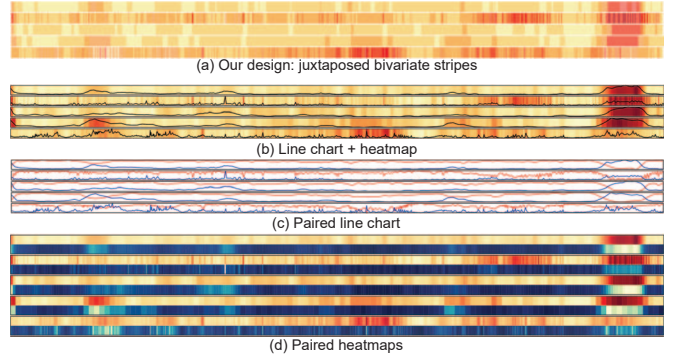


Fig. 6: Our design of juxtaposed bivariate stripes (a) has better scalability than alternative designs: (b) line chart + heatmap, (c) paired line chart, and (d) paired heatmap.

has no intersection with others, effectively eliminating overlapping.

Second, we further employ a max-pooling aggregation method to ensure that metrics in each sliding window are covered. Specifically, each pixel on the horizontal axis represents the aggregation of n_w windows, where $n_w = \lfloor \frac{N_w}{P} \rfloor$, N_w denotes the total number of windows and P denotes the number of available pixels on the horizontal axis. Then, in the max-pooling, the color encoding of the aggregated pixel is determined by the element-wise maximum of the bivariate metrics (e.g., RMSE vs. CORR.) for n_w windows. The max-pooling aggregation can better preserve outliers than the commonly used average aggregation. We can identify some abnormal representations and predictions from the juxtaposed bivariate stripes. As an example, the Raw/Sk-3 and MA-3/Sk-1 representations (Figure 4 (D)) are abnormal representations with large errors for all sliding windows, indicating that they are not suitable representations for this forecasting task.

Alternative Design: As shown in Figure 6, we also considered three alternative design choices, including (b) line chart + heatmap, (c) paired line chart, and (d) paired heatmaps. For all the designs, each juxtaposed row visualizes both the RMSE and CORR. of a specific representation.

Specifically, line chart + heatmap (Figure 6 (b)) uses separate visual channels for different metrics by simply combining a line chart for one metric (e.g., CORR.) with color-encoded background (e.g., RMSE) for the other. However, the background heatmap captures most of the attention due to the strong pre-attentive influence of colors, making it difficult to compare lines in different rows. Paired line chart (Figure 6 (c)) encodes RMSE and CORR. as two separate lines, making it difficult to compare and explore the metrics because their ranges are different. Heatmaps are superior than line charts for displaying time-series data when the display space is limited [46]. Another alternative design is paired heatmaps (Figure 6 (d)) that arrange the bivariate metrics as two separate heatmaps, which would require two times more vertical space than our design using bivariate colormaps.

Therefore, these alternative designs are less scalable than the proposed bivariate juxtaposed stripes, especially when the number of representations increases due to the increasing number of transformation methods and their parameters.

5.5 Prediction Comparator View

This view is designed to reveal the relationship between explanation metrics and performance metrics (T2.3) and subsequently analyze prediction outliers (T3.1). To support the investigation of each prediction, we calculate its corresponding correlation coefficient, prediction error, and SHAP value. In this way, each prediction can be represented as a point in the scatterplot, as shown in Figure 4 (E3). Each point is encoded based on two-dimensional values schemed using the same VSUP with that in *Representation View*.

Due to the large amount of predictions, displaying all of them would cause visual clutter. Therefore, we use random sampling to reduce the overdrawing problem, along with lasso selection allowing users to see more local details by interaction. The random sampling method can faithfully capture the density information of the whole distribution with the number of prediction points in each region proportional to local data density, which is important for the observation of distribution patterns and outliers. To see more details, users can select an interesting region using the lasso selection tool, highlighting all points in the region. In addition, to show a more accurate prediction-level metric relationship, we provide a table (Figure 4 (E2)) adjacent to the scatterplot showing prediction errors (RMSE) along with correlation coefficients. Users can sort the table according to different metrics for multi-faceted exploration. The data filtering operation in the scatterplot and the table is coordinated, with lasso selection linked to row filtering of the table.

5.6 User Interaction

In addition to the coordinated views, *TimeTuner* also integrates various interactions to enable:

- **Selection & Filtering:** In *Variable Inspector View*, users can hover over the interested plot to view more details in the upper right corner. If one plot is selected, the corresponding smoothed time series representation is displayed in the line chart of *Temporal View*. Users can select a specific representation in *Representation View*, predictions of interest in the table of *Prediction Comparator View*, or filter a subset of predictions with the lasso tool in the scatterplot of *Prediction Comparator View*. Selected/filtered prediction points are highlighted with black boundaries, while other points turn gray. In *Representation View*, corresponding stripes are highlighted for users to explore abnormal predictions within the subset selected by the lasso tool. Users can select twice with the lasso tool in *Prediction Comparator View* and get a union of highlighted prediction points.
- **Brushing:** Brushing is supported in *Temporal View* for users to observe detailed changes in time series. When a particular data point is selected in the table of *Prediction Comparator View*, the slider will be automatically moved to the corresponding time window.
- **Sorting:** Users can sort representation or data points by certain metrics in the tables presented in *Profile View* or *Prediction Comparator View*, correspondingly. *TimeTuner* also supports reordering the stripes in *Representation View* by dragging certain stripes side-by-side to promote the comparison between the representations.

- **Scrolling:** *Representation View* supports scrolling for displaying and comparing large representations. Since the view has limited space, displaying more than 30 representations can result in an excessively narrow height for each representation. When selecting multiple transformation methods and parameters, users can compare and explore the representations through a combination of scrolling and sorting.

6 EVALUATION

We conduct case studies — one for univariate (Sect. 6.1) and one for multivariate (Sect. 6.2) time-series forecasting task, and semi-structured expert interviews (Sect. 6.3) to evaluate the feasibility of *TimeTuner*. We illustrate these cases from the perspective of Mike, a system user, seeking a comprehensive understanding of the dataset and optimal transformation methods for the forecasting tasks.

6.1 Case 1: Univariate sunspots forecasting

In this case, Mike selects a famous univariate dataset “Sunspot” that describes a solar phenomenon wherein a dark spot forms on the sun’s surface. The data is collected from the total sunspot number V2.0 published by SILSO. The data include the monthly number of sunspots from January 1, 1818 to November 30, 2022. The forecasting goal is to predict the number of sunspots in the next ten years (120 months) using sunspots from the past sixty years (720 months).

Firstly, Mike begins by selecting the smoothing methods and sampling parameters. He notices the periodicity of the sunspot dataset contains 1 month, 3 months, half a year, and 13 months, so he chooses the MA and WMA methods based on these four parameters and sets these four parameters as the skip length. As Figure 7 shows, by sorting the prominent bar chart in *Profile View*, Mike can identify that the representation ‘WMA-13/Sk-3’ achieves the highest prediction accuracy (Figure 7 (A)) and the representation ‘MA-3/Sk-1’ has the largest training error (Figure 7 (B)). Secondly, he wants to learn why the representation ‘WMA-13/Sk-3’ is better suited than others. So he clicks the bar and the corresponding representation is highlighted in *Representation View*. The representation ‘WMA-13/Sk-3’ is the brightest one with significant SLICS errors only in the interval between 2009 to 2018 (red box). He finds that almost all representations have large SLICS errors in this interval. By observing the horizons in the *Temporal View*, Mike realizes that this might be because 2009 is the period of minimum solar activity (the average number of sunspots in August 2009 was zero, the lowest recorded in 96 years since June 1913) [51], and the model does not capture the weakening of sunspot activity.

Thirdly, Mike wants to select two predictions from different representations to inspect the counterfactual explanations. He clicks on the horizon ‘MA-6’ and ‘WMA-13’ and then obtains the details of smoothed curves (Figure 7 (C) and (D)) in *Temporal View*. By observing the *Prediction Comparator View*, he finds that there exists a negative relationship between CORR. and RMSE, with most of the predictions concentrated in the lower right corner.

For the predictions with obvious errors in the upper left corner (Figure 7 (E)), they correspond to the darkest SLICS in *Representation View* (T2.3). By clicking on one prediction of the selected results, Mike focuses on the prediction values of this outlier window in *Temporal View*, indicating that this presentation is unsuitable for this task. Besides the CORR., Mike changes the X-axis to SHAP value and finds that predictions with small prediction errors are concentrated in the middle part, while those with large prediction errors are concentrated on sides. This situation is because each window’s contribution to predictions is similar for models with good prediction results. Significantly different contributions from some windows may indicate an overfitting problem.

6.2 Case 2: Multivariate PM2.5 forecasting

In this case, Mike selects a multivariate air pollutant dataset, which is collected by the Beijing US Embassy². The data we used include the hourly data from 00:00 on January 1, 2013 to 23:00 on March 31, 2014 (15 months). In this case, the forecasting task is to predict the values of PM2.5 in the next 12 hours using six variables, including

²<https://aqicn.org/city/beijing/us-embassy/cn/>

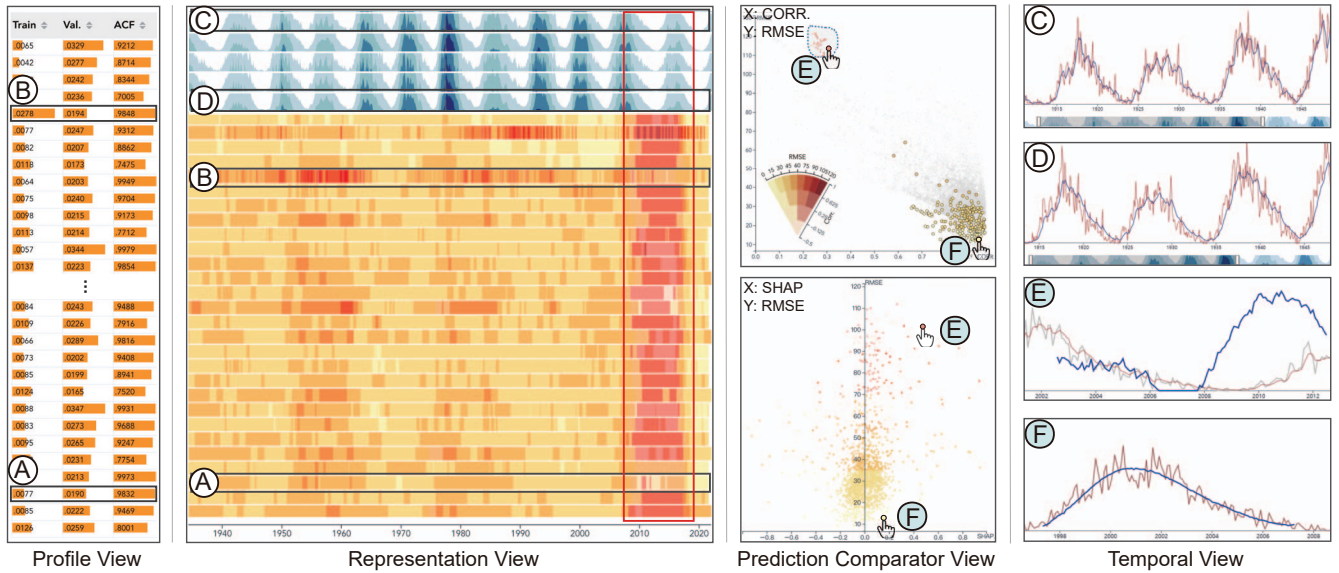


Fig. 7: TimeTuner can help identify abnormal representations and time slices in the case of forecasting sunspots. A, B, C, and D with the white tag are time-series representations, while E and F with the blue tag are focused time slices.

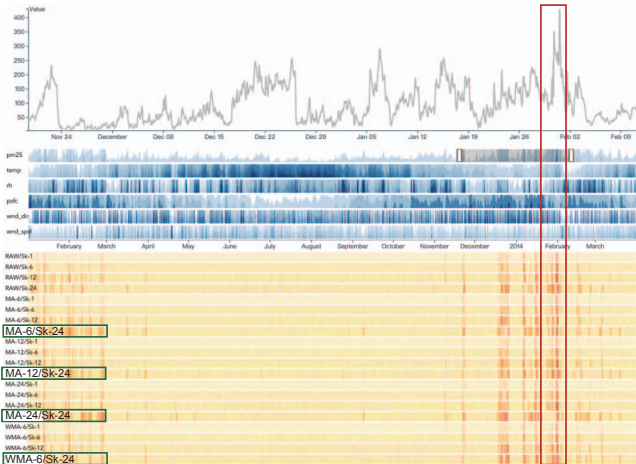


Fig. 8: Compare the effects of different representations on predictions in forecasting PM2.5.

PM2.5, temp (temperature), rh (relative humidity), psfc (SFC pressure), wnd_dir (wind direction), wnd_spd (wind speed) from the past 24 hours. Outliers caused by specific events (such as Chinese New Year) are difficult to be captured and learned by models, resulting in poor performance of all models in the time range.

Firstly, Mike can easily find that the representation ‘WMA-24/Sk-1’ is the most suitable one for the task by the sortable bar chart in *Profile View* (T2.3). He then examines the *Representation View* (Figure 8) and identifies a time interval from January 30, 2014 to February 2, 2014 (red box) where all representations have high prediction errors, which he attributes to the setting off of fireworks during the CNY (T3.1). Secondly, Mike wants to analyze the differences among different sampling methods and determines that the representations of ‘Sk-24’ have poor performances, as shown at the beginning of the timeline (green boxes). After analyzing, Mike thinks it may be because the skip length is too large for these forecasting tasks, which ignores many typical patterns of the input dataset (T2.1).

After exploring different representations, Mike wants to explore the relationship between different variable/variable pairs for PM2.5. He begins exploration with *Prediction Inspector View* (Figure 9), and finds that there exists a negative correlation between PM2.5 and temperature (Figure 9 (A)), and a positive correlation with relative humidity. Then he observes the horizon graph of PM2.5, temperature, and relative

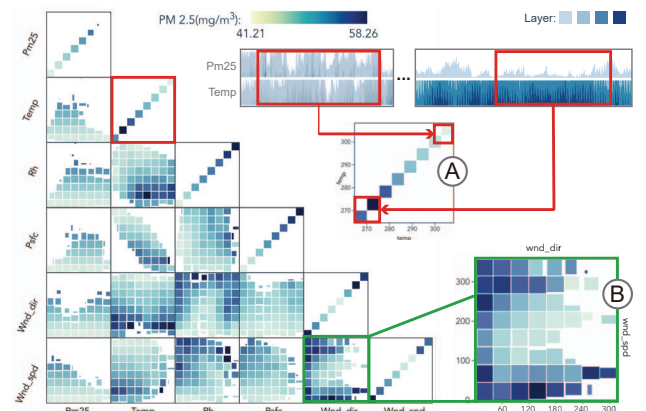


Fig. 9: Identify the interactions among variables in the PM2.5 dataset.

humidity and finds that the temperature is high in summer and the PM2.5 is low (red boxes), whilst PM2.5 is high in winter. Through inquiries, Mike learns that this may be due to two points: 1) *Pollutant emissions*. In winter, coal is mostly used for heating in the northern region, and the amount of coal burned increases significantly. What’s more, due to the decrease in temperature in winter, the gas pressure and temperature of the working cycle of the automobile engine are not high, the combustion rate of the mixed gas slows down, causing incomplete combustion, and the exhaust emissions of motor vehicles increase, resulting in an increase in the emission of PM2.5 and its precursors (e.g., sulfur dioxide, nitrogen oxides, VOCs). 2) *Temperature inversion*. At night in autumn and winter, the ground temperature drops sharply, so the temperature of the lower atmosphere close to the ground is very low, while the upper air does not cool down so quickly. The temperature of the high layer is higher than that of the low layer, and the phenomenon of ‘temperature reversal’ occurs. Once this cold and hot inversion layer is formed, the air cannot be convectioned up and down, making it difficult for pollutants to diffuse [36].

Besides the temperature and PM2.5 relations, Mike discovers intriguing insights in the ‘Wind direction - Wind speed’ plot (Figure 9 (B)). Low wind speeds hinder contaminant dispersion, resulting in high PM2.5 concentrations. Surprisingly, even with high wind speeds, regions with south winds still exhibit elevated PM2.5 levels. Mike speculates that the anomaly may stem from significant pollutant sources like factories and coal-fired power plants in the southern areas of Beijing.

6.3 Expert Interview

We conducted one-on-one interviews with five DL experts (*DL1-DL5*) and five visualization experts (*VE1-VE5*). All DL experts have experience in developing time-series forecasting models and can offer valuable feedback on visualization systems. The five visualization experts all have experience developing and working with interactive visualization systems and can provide valuable feedback on visual and interaction design from users' perspectives. The interviews were held in a semi-structured format. We first introduced the workflow of *TimeTuner*. Then, we introduced the interface of *TimeTuner* using the case examples from Sect. 6.1 and 6.2. The introductory session took about thirty minutes. Afterward, we asked them to explore *TimeTuner* freely for fifteen minutes. Finally, we collected their feedback.

Usability Experts (*DL1-DL5*) commended the combination of counterfactual explanations with visual analytics for analyzing the impact of time-series representations on LSTM-based predictions. They acknowledged that time-series representations are as significant as deep learning models but have yet to receive more research attention. All experts (*DL1-DL5*) praised the system for its ability to support the exploration of both univariate and multivariate data. According to a DL expert, "*the system helps me understand the input time-series data easily and select the appropriate time-series representations intuitively*". Most experts agreed that it is reasonable to divide the overall goal into tasks at variable-, representation-, and prediction-level tasks, which *TimeTuner* successfully accomplished. However, *DL2* had some confusion with the VSUP-based juxtaposed bivariate stripes, where she cannot identify sliding windows with smaller RMSE. After shown the stripes only for the RMSE metric in *Representation View*, she appreciated that *TimeTuner* provided unique insights and a complete exploration process for time-series forecasting tasks.

Visual and Interaction Design. *TimeTuner's* visual and interaction design received positive feedback from both DL and visualization experts. They found the views intuitive and well-aligned with data and analytical tasks. The DL experts expressed particular enthusiasm for the horizon graphs and juxtaposed bivariate stripes, which were new to them. *DL3* commented: "*These charts look impressive and have practical value in visualizing large-scale time series.*" Some visualization experts (*VE1* & *VE3*) were unfamiliar with the VSUP palette [22] but appreciated its effectiveness over conventional bivariate colormaps in simultaneously presenting input features and prediction errors. Three experts (*DL2*, *DL3* & *DL5*) praised the selection and filtering operations, which helped them more effectively explore the relationship between representations and abnormal predictions. Two experts (*VE4* & *VE5*) found *Variable Inspector View* slightly confusing for visualizing univariate time-series data. After we clarified the meaning of each axis and color encoding, they were able to understand how to compare smoothing methods for univariate time series through this view.

7 DISCUSSION

The case studies demonstrate how *TimeTuner* aids users in comprehending the impact of minimal solar activity on sunspot forecasting (Sect. 6.1), as well as the adverse effect of temperature on PM2.5 prediction (Sect. 6.2). This is accomplished by providing counterfactual explanations for time-series representations and an interactive visualization system that enables users to investigate the connections among variables, representations, and predictions from different perspectives. Domain experts verified the effectiveness of *TimeTuner* in selecting appropriate representations for DL-based time-series forecasting.

Applicability. *TimeTuner* is a versatile system that can be easily applied to other scenarios and DL models for time-series forecasting. This is because *TimeTuner* is model-agnostic and focuses on input representations and output predictions. Specifically, we utilized smoothing and sampling methods commonly used for time-series transformation. This process can be extended effortlessly to other transformation methods, including padding missed values, selecting features, etc., as suggested by a domain expert. We used explanation metrics such as SHAP and CORR., which also apply to other tasks, including time-series classification and clustering. Additional explanation metrics can also be inte-

grated, such as partial dependence [25] and summation-to-delta [54].

Scalability. Currently, *TimeTuner* relies on SHAP, which is time-consuming when calculating correlations among time slices, variables, and predictions. As a result, all counterfactual explanation metrics are precomputed. While some experts are willing to wait, allowing users to add new parameters and show the results interactively would be preferable. This could be achieved using less computationally intensive metrics or implementing parallel computing. From the visualization perspective, *TimeTuner* is scalable to handle large-volume time series, benefiting from the innovative design of the juxtaposed bivariate stripes. This design combines pixel-wise heat stripes with VSUP to achieve seamless encoding of bivariate values in time-series representations and predictions. However, scalability issues can still arise when dealing with too many variables and representations of large-scale, large-volume time-series data. When the user selects too many transformation methods and parameters in the control panel or the selected dataset has a large number of variables, the massive elements can cause the system to crash. Our informal scalability testing suggests a minimum width of 20 pixels per plot/horizon/representation. It means that the variable number/smoothing methods should not exceed 9, and the number of representations should not exceed 30, otherwise may lead to visual disorder and difficult interactions. To address this issue, we can use more robust preprocessing methods such as progressive clustering [58] to combine similar time series and implement sophisticated visual and interaction designs, such as multi-foci navigation [67].

Limitations and Future Work. *TimeTuner* currently has several limitations that need to be addressed in future work. First, our design of the *Representation View* and *Prediction Comparator View* may lead to confirmation bias because we use a backward-oriented reasoning process that acquires the diagnostic predictions first and then looks for supporting evidence. To mitigate this issue, we need to investigate whether our visualization designs may alleviate potential risks and whether cognitive biases affect the experts' decisions. This can be done by conducting user studies to assess the precision of users' selection and cognition when using *TimeTuner*. Second, we offer limited time-step parameters in the sunspot and PM2.5 forecasting tasks, which may not be enough for some other scenarios. To test the universality of *TimeTuner* on more types of time-series data and models, we need to develop more computation-efficient metrics, robust preprocessing methods, and sophisticated visual and interaction designs to address scalability issues. Furthermore, generating counterfactual explanations for various types of data and questions is an active research problem that we aim to support in the future. We plan to open-source the code for time-series transformation and the visualization system to promote future research in this direction.

8 CONCLUSION

This work highlighted three key challenges that limit the effectiveness of automated approaches for representation learning in time-series forecasting: the limitation of incorporating prior knowledge, identifying interactions among variables, and choosing evaluation metrics to ensure the model's reliability. To overcome these limitations, we introduced a general visual analytics framework called *TimeTuner*, which can help analysts understand model behaviors and associations at the variable, representation, and prediction levels. We demonstrated the effectiveness of our approach in terms of in-depth insights learned in two case studies and positive feedback received from expert interviews. Our findings suggest that visual association with localized correlations, stationarity, and different time-series representations significantly improve users' ability to interpret model performances and improve engineering processes. Overall, our work has highlighted the significant potential of counterfactual explanations combined with visual analytics in advancing the field of data representation learning.

SUPPLEMENTAL MATERIALS

All source code of *TimeTuner* is freely available at <https://github.com/CatherineHao/TimeTuner>.

ACKNOWLEDGMENTS

The authors wish to thank anonymous reviewers for their constructive comments. This work was supported in part by the National Natural Science Foundation of China (No. 62172398) and Guangdong Basic and Applied Basic Research Foundation (2021A1515011700).

REFERENCES

- [1] Flask documentation. <https://flask.palletsprojects.com/>, last accessed on 29/06/2023. 3
- [2] The javascript library for bespoke data visualization. <https://d3js.org/>, last accessed on 29/06/2023. 3
- [3] Keras. <https://keras.io>, last accessed on 29/06/2023. 4
- [4] scipy.stats.pearsonr. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html>, last accessed on 29/06/2023. 5
- [5] Vue.js. <https://v2.vuejs.org/>, last accessed on 29/06/2023. 3
- [6] Welcome to the shap documentation. <https://shap.readthedocs.io/>, last accessed on 29/06/2023. 4
- [7] A. Adadi and M. Berrada. Peeking inside the black-box: a survey on explainable artificial intelligence (xai). *IEEE Access*, 6:52138–52160, 2018. doi: 10.1109/ACCESS.2018.2870052 2
- [8] Y. Ahn and Y. Lin. FairSight: visual analytics for fairness in decision making. *IEEE Trans. Vis. Comput. Graph.*, 26(1):1086–1095, 2019. doi: 10.1109/TVCG.2019.293426 3
- [9] W. Aigner, S. Miksch, H. Schumann, and C. Tominski. *Visualization of time-oriented data*, vol. 4. Springer, 2011. doi: 10.1007/978-0-85729-079-3 2
- [10] W. Aigner, S. Miksch, B. Thurnher, and S. Biffl. Planninglines: novel glyphs for representing temporal uncertainties and their evaluation. In *Proc. IEEE IV*, pp. 457–463, 2005. doi: 10.1109/IV.2005.97 2
- [11] A. G. Asuero, A. Sayago, and A. González. The correlation coefficient: An overview. *Crit. Rev. Anal. Chem.*, 36(1):41–59, 2006. doi: 10.1080/10408340500526766 5
- [12] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE Trans. Pattern Anal. Mach. Intell.*, 35(8):1798–1828, 2013. doi: 10.1109/TPAMI.2013.50 1, 2, 3
- [13] R. Binns, M. Kleek, M. Veale, U. Lyngs, J. Zhao, and N. Shadbolt. ‘it’s reducing a human being to a percentage’ perceptions of justice in algorithmic decisions. In *Proc. ACM CHI*, pp. 1–14, 2018. doi: 10.1145/3173574.3173951 2
- [14] G. E. Box and D. A. Pierce. Distribution of residual autocorrelations in autoregressive-integrated moving average time series models. *J. Am. Stat. Assoc.*, 65(332):1509–1526, 1970. doi: 10.1080/01621459.1970.10481180 2
- [15] M. Brehmer, B. Lee, B. Bach, N. H. Riche, and T. Munzner. Timelines revisited: A design space and considerations for expressive storytelling. *IEEE Trans. Vis. Comput. Graph.*, 23(9):2151–2164, 2016. doi: 10.1109/TVCG.2016.2614803 2
- [16] M. Burch, A. Veneri, and B. Sun. EyeClouds: A visualization and analysis tool for exploring eye movement data. In *Proc. VINCI*, pp. 1–8, 2019. doi: 10.1145/3356422.3356423 2
- [17] L. Byron and M. Wattenberg. Stacked graphs—geometry & aesthetics. *IEEE Trans. Vis. Comput. Graph.*, 14(6):1245–1252, 2008. doi: 10.1109/TVCG.2008.166 2
- [18] G. Cascarino, M. Moscatelli, and F. Parlapiano. Explainable artificial intelligence: interpreting default forecasting models based on machine learning. *Bank of Italy Occasional Paper*, (674), 2022. doi: 10.32057/0.QEF.2022.0674 3
- [19] F. Cheng, Y. Ming, and H. Qu. DECE: Decision explorer with counterfactual explanations for machine learning models. *IEEE Trans. Vis. Comput. Graph.*, 27(2):1438–1447, 2020. doi: 10.1109/TVCG.2020.3030342 2, 3
- [20] Y.-W. Cheung and K. S. Lai. Lag order and critical values of the augmented Dickey–Fuller test. *J. Bus. Econ. Stat.*, 13(3):277–280, 1995. doi: 10.2307/1392187 4
- [21] R. Chimmula and L. Zhang. Time series forecasting of covid-19 transmission in canada using lstm networks. *Chaos Solit. Fractals*, 135:109864, 2020. doi: 10.1016/j.chaos.2020.109864 1, 2
- [22] M. Correll, D. Moritz, and J. Heer. Value-suppressing uncertainty palettes. In *Proc. ACM CHI*, 2018. doi: 10.1145/3173574.3174216 2, 6, 9
- [23] R. Dachsel and M. Weiland. Timezoom: a flexible detail and context timeline. In *Proc. ACM CHI*, pp. 682–687, 2006. doi: 10.1145/1125451.1125590 2
- [24] S. Elsayed, D. Thyssens, A. Rashed, S. Jomaa, and L. Thieme. Do we really need deep learning models for time series forecasting? *arXiv*, 2021. doi: 10.48550/arXiv.2101.02118 1, 2
- [25] J. H. Friedman. Greedy function approximation: a gradient boosting machine. *Ann. Stat.*, pp. 1189–1232, 2001. doi: 10.1214/aos/1013203451 9
- [26] E. S. Gardner Jr. Exponential smoothing: The state of the art. *J. Forecast.*, 4(1):1–28, 1985. doi: 10.1002/for.3980040103 2
- [27] M. C. Hao, U. Dayal, D. A. Keim, and T. Schreck. Multi-resolution techniques for visual exploration of large time-series data. In *Proc. EUROVIS*, pp. 27–34, 2007. doi: 10.2312/VisSym/EuroVis07/027-034 2
- [28] J. Heer, N. Kong, and M. Agrawala. Sizing the horizon: the effects of chart size and layering on the graphical perception of time series visualizations. In *Proc. ACM CHI*, pp. 1303–1312, 2009. doi: 10.1145/1518701.1518897 1, 5
- [29] W. Javed, B. McDonnell, and N. Elmqvist. Graphical perception of multiple time series. *IEEE Trans. Vis. Comput. Graph.*, 16(6):927–934, 2010. doi: 10.1109/TVCG.2010.162 2
- [30] A.-H. Karimi, G. Barthe, B. Balle, and I. Valera. Model-agnostic counterfactual explanations for consequential decisions. In *Proc. AISTATS*, pp. 895–905. PMLR, 2020. 2
- [31] R. Kincaid. Signallens: Focus + context applied to electronic time series. *IEEE Trans. Vis. Comput. Graph.*, 16(6):900–907, 2010. doi: 10.1109/TVCG.2010.193 2
- [32] R. Kincaid and H. Lam. Line graph explorer: scalable display of line graphs using focus+ context. In *Proc. ACM AVI*, pp. 404–411, 2006. doi: 10.1145/1133265.1133348 2
- [33] B. C. Kwon, M. Choi, J. T. Kim, E. Choi, Y. B. Kim, S. Kwon, J. Sun, and J. Choo. Retainvis: Visual analytics with interpretable and interactive recurrent neural networks on electronic medical records. *IEEE Trans. Vis. Comput. Graph.*, 25(1):299–309, 2019. doi: 10.1109/TVCG.2018.2865027 2
- [34] T. Lammarsch, W. Aigner, A. Bertone, J. Gärtner, E. Mayr, S. Miksch, and M. Smuc. Hierarchical temporal patterns and interactive aggregated views for pixel-based visualizations. In *Proc. IEEE ICIV*, pp. 44–50, 2009. doi: 10.1109/IV.2009.52 2
- [35] D. Lewis. *Counterfactuals*. 1973. doi: 10.2307/2273738 2
- [36] X. Liang, T. Zou, B. Guo, S. Li, H. Zhang, S. Zhang, H. Huang, and S. X. Chen. Assessing beijing’s pm2.5 pollution: severity, weather impact, apec and winter heating. *Proc. Math. Phys. Eng. Sci. P ROY SOC A-MATH PHY*, 471(2182):20150257, 2015. doi: 10.1098/rspa.2015.0257 8
- [37] B. Lim, S. Zohren, and S. Roberts. Recurrent neural filters: Learning independent bayesian filtering steps for time series prediction. In *Proc. IEEE IJCNN*, pp. 1–8. IEEE, 2020. doi: 10.1109/IJCNN48605.2020.9206906 2
- [38] B. Lindemann, T. Müller, H. Vietz, N. Jazdi, and M. Weyrich. A survey on long short-term memory networks for time series prediction. *Procedia CIRP*, 99:650–655, 2021. doi: 10.1016/j.procir.2021.03.088 2
- [39] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, eds., *Proc. NIPS*, pp. 4765–4774. 2017. doi: 10.5555/3295222.3295230 4
- [40] Y. Ming, P. Xu, F. Cheng, H. Qu, and L. Ren. Protosteer: Steering deep sequence model with prototypes. *IEEE Trans. Vis. Comput. Graph.*, 26(1):238–248, 2020. doi: 10.1109/TVCG.2019.2934267 2
- [41] P. Mishra. *Counterfactual Explanations for XAI Models*, pp. 265–278. Berkeley, CA, 2022. doi: 10.1145/3514094.3534144 2
- [42] C. Molnar. *Interpretable machine learning*. Lulu. com, 2020. 3
- [43] R. K. Mothilal, A. Sharma, and C. Tan. Explaining machine learning classifiers through diverse counterfactual explanations. In *Proc. ACM FAccT*, pp. 607–617, 2020. doi: 10.1145/3351095.3372850 2
- [44] T. Mühlbacher and H. Piringer. A partition-based framework for building and validating regression models. *IEEE Trans. Vis. Comput. Graph.*, 19(12):1962–1971, 2013. doi: 10.1109/TVCG.2013.125 1, 2, 3, 5
- [45] T. Nocke, H. Schumann, U. Böhm, and M. Flechsig. Information visualization supporting modelling and evaluation tasks for climate models. In *Proc. WSC*, vol. 1, pp. 763–771, 2003. doi: 10.1109/WSC.2003.1261493 2
- [46] M. Oppermann, L. Liu, and T. Munzner. Timeelide: Visual analysis of non-contiguous time series slices. In *Proc. IEEE VIS*, pp. 41–45. IEEE, 2021. doi: 10.1109/VIS49827.2021.9623320 7
- [47] S. S. Rangapuram, M. W. Seeger, J. Gasthaus, L. Stella, Y. Wang, and T. Januschowski. Deep state space models for time series forecasting. *Adv.*

- Condens. Matter Phys.*, 31, 2018. doi: 10.5555/3327757.3327876 2
- [48] A. Sagheer and M. Kotb. Time series forecasting of petroleum production using deep lstm recurrent networks. *Neurocomputing*, 323:203–213, 2019. doi: 10.1016/j.neucom.2018.09.082 1, 2
- [49] T. Saito, H. N. Miyamura, M. Yamamoto, H. Saito, Y. Hoshiya, and T. Kaseda. Two-tone pseudo coloring: Compact visualization for one-dimensional data. In *Proc. IEEE InfoVis*, pp. 173–180. IEEE, 2005. doi: 10.1109/INFVIS.2005.1532144 2
- [50] S. Sarkar, T. Weyde, A. d. Garcez, G. G. Slabaugh, S. Dragicevic, and C. Percy. Accuracy and interpretability trade-offs in machine learning applied to safer gambling. In *Proc. CEUR*, vol. 1773, 2016. 2
- [51] C. J. Schrijver, W. Livingston, T. Woods, and R. Mewaldt. The minimal solar activity in 2008–2009 and its implications for long-term climate modeling. *Geophysical Research Letters*, 38(6), 2011. doi: 10.1029/2011GL046658 7
- [52] Q. Shen, Y. Wu, Y. Jiang, W. Zeng, A. K. H. LAU, A. Vianova, and H. Qu. Visual interpretation of recurrent neural network on multi-dimensional time-series forecast. In *Proc. IEEE PacificVis*, pp. 61–70, 2020. doi: 10.1109/PacificVis48177.2020.2785 2, 3
- [53] Y. Shi, B. Chen, Y. Chen, Z. Jin, K. Xu, X. Jiao, T. Gao, and N. Cao. Supporting guided exploratory visual analysis on time series data with reinforcement learning. *IEEE Trans. Vis. Comput. Graph.*, pp. 1–10, 2023. 2
- [54] A. Shrikumar, P. Greenside, and A. Kundaje. Learning important features through propagating activation differences. In *Proc. ICML*, p. 3145–3153, 2017. doi: 10.5555/3305890.3306006 9
- [55] S. Siami-Namini and A. S. Namin. Forecasting economics and financial time series: Arima vs. lstm. *arXiv*, 2018. doi: 10.48550/arXiv.1803.06386 1, 2
- [56] T. Spinner, U. Schlegel, H. Schäfer, and M. El-Assady. explAiner: A visual analytics framework for interactive and explainable machine learning. *IEEE Trans. Vis. Comput. Graph.*, 26(1):1064–1074, 2019. doi: 10.1109/TVCG.2019.2934629 3
- [57] H. Strobelt, S. Gehrmann, H. Pfister, and A. M. Rush. Lstmvis: A tool for visual analysis of hidden state dynamics in recurrent neural networks. *IEEE Trans. Vis. Comput. Graph.*, 24(1):667–676, 2018. doi: 10.1109/TVCG.2017.2744158 2
- [58] J. J. Van Wijk and E. R. Van Selow. Cluster and calendar based visualization of time series data. In *Proc. IEEE InfoVis*, pp. 4–9. IEEE, 1999. doi: 10.1109/INFVIS.1999.801851 2, 9
- [59] S. Verma, V. Boonsanong, M. Hoang, K. E. Hines, J. P. Dickerson, and C. Shah. Counterfactual explanations and algorithmic recourses for machine learning: A review. *arXiv*, 2020. doi: 10.48550/arXiv.2010.10596 2
- [60] Y. Wang, F. Han, L. Zhu, O. Deussen, and B. Chen. Line graph or scatter plot? automatic selection of methods for visualizing trends in time series. *IEEE Trans. Vis. Comput. Graph.*, 24(2):1141–1154, 2017. doi: 10.1109/TVCG.2017.2653106 2
- [61] J. Wexler, M. Pushkarna, T. Bolukbasi, M. Wattenberg, F. Viégas, and J. Wilson. The what-if tool: Interactive probing of machine learning models. *IEEE Trans. Vis. Comput. Graph.*, 26(1):56–65, 2020. doi: 10.1109/TVCG.2019.2934619 1, 2, 3
- [62] C. Willmott and K. Matsuura. Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Clim. Res.*, 30(1):79–82, 2005. doi: 10.3354/cr030079 4
- [63] K. Xu, S. Guo, N. Cao, D. Gotz, A. Xu, H. Qu, Z. Yao, and Y. Chen. Ecglens: Interactive visual exploration of large scale ecg data for arrhythmia detection. In *Proc. ACM CHI*, pp. 1–12, 2018. doi: 10.1145/3173574.3174237 2
- [64] K. Xu, J. Yuan, Y. Wang, C. Silva, and E. Bertini. Mtseer: Interactive visual exploration of models on multivariate time-series forecast. In *Proc. ACM CHI*, pp. 1–15, 2021. doi: 10.1145/3411764.3445083 2
- [65] M. Yajnik, S. Moon, J. Kurose, and D. Towsley. Measurement and modelling of the temporal dependence in packet loss. In *Proc. IEEE INFOCOM*, vol. 1, pp. 345–352. IEEE, 1999. doi: 10.1109/INFCOM.1999.749301 4
- [66] W. Zeng, C. Lin, J. Lin, J. Jiang, J. Xia, C. Turkay, and W. Chen. Revisiting the modifiable areal unit problem in deep traffic prediction with visual analytics. *IEEE Trans. Vis. Comput. Graph.*, 27(2):839–848, 2020. doi: 10.1109/TVCG.2020.3030410 2
- [67] J. Zhao, F. Chevalier, and R. Balakrishnan. KronoMiner: Using multi-foci navigation for the visual exploration of time-series data. In *Proc. ACM CHI*, pp. 1737–1746, 2011. doi: 10.1145/1978942.1979195 9